

CSP/NOI 模拟试卷解析

一、单项选择题（每题 2 分）

第 1 题

题目：在 NOI Linux 系统中，使用 g++ 编译 C++ 程序时，如果要生成可调试的可执行文件，应该使用选项

选项：-O2 / -g / -static / -Wall

答案：B -g

解析：-g 选项生成调试信息，供 GDB 等调试器使用。

第 2 题

题目：执行 `int x = 0xAB; cout << (x >> 4 | (x & 0x0F) << 4);`，输出为

选项：186 / 171 / 180 / 176

答案：A 186

解析： $0xAB = 10101011_2$ ， $x \gg 4 = 0x0A$ ， $(x \& 0x0F) \ll 4 = 0xB0$ ，按位或得 $0xBA = 186$ 。

第 3 题

题目：连接 n 个字符串，每个字符串的长度不大于 m ，最坏情况下时间复杂度是

选项： $O(nm)$ / $O(n^2m)$ / $O(nm^2)$ / $O(\log n)$

答案：B $O(n^2m)$

解析：朴素拼接每次需复制已连接部分，总复制量约为 $m(1 + 2 + \cdots + n) = O(n^2m)$ 。

第 4 题

题目：已知某完全二叉树的层次遍历序列为 ACBGFED，则其中序遍历序列是

选项：DBEAFCEG / ABDCEFG / GCFAEBD / GCFAEDB

答案：C GCFAEBD

解析：按层建树后中序遍历（左-根-右）得到 G C F A E B D。

第 5 题

题目：用两个栈模拟队列，如果 push 操作总是入栈 S_1 ，那么 pop 操作是

选项：

A. 直接弹出 S_1 栈顶

B. 直接弹出 S_2 栈顶

C. 将 S_2 全部弹出并压入 S_1 ，然后弹出 S_1 栈顶

D. 如果 S_2 为空，将 S_1 全部弹出并压入 S_2 ，然后弹出 S_2 栈顶

答案：D

解析：经典双栈模拟队列： S_1 负责入队， S_2 负责出队；仅当 S_2 为空时才将 S_1 倒入 S_2 。

第 6 题

题目：关于树的重心，下列说法中不正确的是

选项：

- A. 树至少有一个重心
- B. 树最多有两个重心，且这两个重心一定相邻
- C. 树最多有两个重心，且这两个重心可以不相邻
- D. 重心到其他点的距离之和最小

答案：C

解析：树的重心有 1 个或 2 个；若有两个，它们**必定相邻**。因此"可以不相邻"是错误的。

第 7 题

题目：给定数字 0、1、5、6、7、9，每个数字最多用一次，可能组成多少个正偶数

选项：216 / 480 / 586 / 589

答案：C 586

解析：按位数分类（末位为 0 或 6）：

1 位：1 个；2 位：9 个；3 位：36 个；4 位：108 个；5 位：216 个；6 位：216 个。

总计 $1 + 9 + 36 + 108 + 216 + 216 = 586$ 。

第 8 题

题目：最长公共子序列（LCS）的应用不包括

选项：文件差异比较 / DNA 序列比对 / 拼写检查 / 数据加密

答案：D 数据加密

解析：LCS 常用于 diff、生物序列比对、拼写检查等，与数据加密无关。

第 9 题

题目：给一个无向图着色，相邻节点不能同色，如果至少需要 3 种颜色，则该图

选项：一定是二分图 / 一定包含奇环 / 一定是完全图 / 一定是连通图

答案：B 一定包含奇环

解析：二分图等价于 2-可着色，也等价于不含奇环。需要 3 种颜色说明不是二分图，必含奇环。

第 10 题

题目：同余方程组 $x \equiv 1 \pmod{3}$, $x \equiv 2 \pmod{5}$ 的解为

选项： $x \equiv 7 \pmod{15}$ / $x \equiv 8 \pmod{15}$ / $x \equiv 11 \pmod{15}$ / $x \equiv 13 \pmod{15}$

答案：A $x \equiv 7 \pmod{15}$

解析：设 $x = 3k + 1$ ，代入得 $3k \equiv 1 \pmod{5}$ ，解得 $k \equiv 2 \pmod{5}$ ，故 $x = 15m + 7$ 。

第 11 题

题目：下列几种排序算法中，在平均情况下，哪一种的时间复杂度与其他三种不同

选项：选择排序 / 堆排序 / 归并排序 / 快速排序

答案：A 选择排序

解析：选择排序平均 $O(n^2)$ ；其余三种平均均为 $O(n \log n)$ 。

第 12 题

题目：5 名同学错位排列（每人不坐自己位置），共有多少种坐法

选项：36 / 44 / 48 / 120

答案：B 44

解析：错位排列 $!5 = 44$ 。

第 13 题

题目：哈希表长度 13， $H(key) = key \bmod 13$ ，线性探查。已插入 {26, 39, 52, 65, 78, 91, 104, 117}，查找 91 的探查下标依次是

选项：0, 1, 2, 3, 4 / 0, 1, 2, 3, 4, 5 / 0, 1, 2, 3, 4, 5, 6 / 0, 1, 2, 3, 4, 5, 6, 7

答案：B 0, 1, 2, 3, 4, 5

解析：插入后位置：0→26, 1→39, 2→52, 3→65, 4→78, 5→91。从 0 开始线性探查，到 5 找到。

第 14 题

题目：以下代码执行后输出

```
int a = 0, b = 1;
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 1) a = a + b;
    else b = a + b;
}
cout << a << " " << b << endl;
```

选项：5 8 / 8 13 / 13 21 / 21 34

答案：B 8 13

解析：模拟得：i=1(1,1), 2(1,2), 3(3,2), 4(3,5), 5(8,5), 6(8,13)。

第 15 题

题目：抛掷一枚均匀骰子，直到出现 2 次 6 为止，期望的抛掷次数是

选项：6 / 12 / 24 / 48

答案：B 12

解析：负二项分布，成功概率 $p = 1/6$ ，需 $r = 2$ 次成功，期望 $r/p = 12$ 。

二、阅读程序

程序一（第 16~20 题）：Legendre 公式求阶乘素因子幂次

```
#include <iostream>
using namespace std;
int main() {
    int n, p;
    cin >> n >> p;
    int ans = 0;
    long long cur = p;
    while (cur <= n) {
        ans += n / cur;
        cur *= p;
    }
    cout << ans << endl;
    return 0;
}
```

16. 输入 10 2, 输出为 8。

答案: A 正确 (2 分)

解析: $\lfloor 10/2 \rfloor + \lfloor 10/4 \rfloor + \lfloor 10/8 \rfloor = 5 + 2 + 1 = 8$ 。

17. 输出一定是正整数。

答案: B 错误 (2 分)

解析: 若 $n < p$ (如 $n=1, p=2$), 循环不执行, 输出 0。

18. 程序的时间复杂度为 $O(\log_p n)$ 。

答案: A 正确 (2 分)

解析: cur 每次乘 p , 循环次数为 $\log_p n$ 。

19. 若希望从低位到高位依次输出 n 在 p 进制下的各位, 应改为

答案: D `cout << n * p / cur % p << endl;` (3 分)

解析: cur 依次取 $p, p^2, \dots$ 。 $n * p / cur$ 等价于 $n / (cur/p)$, 当 $cur=p$ 时得到 $n \bmod p$ (最低位), 随后依次得到更高位。

20. 输入 $n=100, p=5$, 输出为

答案: B 24 (3 分)

解析: $\lfloor 100/5 \rfloor + \lfloor 100/25 \rfloor = 20 + 4 = 24$ 。

程序二 (第 21~25 题): 线性筛预处理约数和

```
#include <iostream>
using namespace std;

const int N = 100009;
int prime[N], tot, sd[N], sp[N];
bool vis[N];

void init() {
    sd[1] = 1;
    for (int i = 2; i < N; i++) {
```

```

        if (!vis[i]) {
            prime[++tot] = i;
            sd[i] = i + 1;
            sp[i] = i + 1;
        }
        for (int j = 1; j <= tot && i * prime[j] < N; j++) {
            vis[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                sp[i * prime[j]] = sp[i] * prime[j] + 1;
                sd[i * prime[j]] = sd[i] / sp[i] * sp[i * prime[j]];
                break;
            } else {
                sp[i * prime[j]] = prime[j] + 1;
                sd[i * prime[j]] = sd[i] * sd[prime[j]];
            }
        }
    }
}

int main() {
    init();
    int n;
    cin >> n;
    cout << sd[n] << endl;
    return 0;
}

```

21. 输入 7，输出是 8。

答案：A 正确 (2 分)

解析：7 是素数，约数和 $1 + 7 = 8$ 。

22. vis[i] 为 1 时，表示 i 是素数。

答案：B 错误 (2 分)

解析：vis[i] 为 true 表示 i 是合数（已被标记）。

23. sd 记录的是所有素因数之和。

答案：B 错误 (2 分)

解析：sd 记录的是所有约数之和 $\sigma(n)$ ，不是素因数之和。

24. 输入 2026 时，输出为

答案：D 3042 (4 分)

解析： $2026 = 2 \times 1013$ (1013 为素数)， $\sigma(2026) = (1 + 2)(1 + 1013) = 3 \times 1014 = 3042$ 。

25. init() 的时间复杂度是

答案：A $O(n)$ (4 分)

解析：线性筛每个合数仅被其最小质因子筛去一次，复杂度 $O(n)$ 。

程序三（第 26~30 题）：费马小定理求逆元

```

#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll T, a, m, ans;

ll quickpow(ll a, ll b) {
    if (b < 0) return 0;
    ll ret = 1;
    a %= m;
    while (b) {
        if (b & 1) ret = (ret * a) % m;
        b >>= 1;
        a = (a * a) % m;
    }
    return ret;
}

ll inverse(ll a, ll m) {
    return quickpow(a, m - 2);
}

int main() {
    cin >> a >> m;
    ans = inverse(a, m);
    cout << ans << " ";
    return 0;
}

```

26. 输入 $a=3$, $m=7$, 输出是 5。

答案：A 正确 (2 分)

解析： $3^5 \bmod 7 = 243 \bmod 7 = 5$ 。

27. 输入 $a=4$, $m=6$, 输出是 4。

答案：A 正确 (2 分)

解析：程序计算 $4^{6-2} = 4^4 \bmod 6$ 。快速幂过程中 $4^2 \equiv 4 \pmod{6}$ ，故 $4^4 \equiv 4 \pmod{6}$ ，程序确实输出 4。（注：虽然 $\gcd(4, 6) \neq 1$ 导致逆元不存在，但题目仅判断输出值。）

28. 输入 $m=2$, 无论 a 为何值，输出都是 1。

答案：A 正确 (2 分)

解析： $m = 2$ 时指数 $m - 2 = 0$ ，快速幂循环不执行，直接返回 $\text{ret} = 1$ 。

29. 程序的时间复杂度是

答案：D $O(\log m)$ (4 分)

解析：快速幂复杂度取决于指数 $m - 2$ 的二进制位数。

30. 关于 `inverse()` 函数，说法正确的是

答案：C 它使用费马小定理，要求 m 必须是素数 (4 分)

解析： $a^{m-2} \equiv a^{-1} \pmod{m}$ 是费马小定理的直接推论，要求 m 为素数且 $a \not\equiv 0 \pmod{m}$ 。

三、完善程序

程序一（第 31~35 题）：树的欧拉序（Euler Tour）

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1009;
vector<int> to[N];
int n, timer, euler[①];

void dfs(int u, int fa) {
    ++timer;
    ②
    for (int i = 0; i < to[u].size(); ++i)
        if (to[u][i] != fa) dfs(to[u][i], u);
    ③
}

int main() {
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        to[u].push_back(v);
        to[v].push_back(u);
    }
    for (int u = 1; u <= n; ++u)
        ④
    dfs(1, 0);
    for (int i = 1; ⑤; ++i) cout << euler[i] << " ";
    cout << euler[2 * n - 1] << endl;
    return 0;
}
```

31. ① 处应填

答案：D $N + N$ (3 分)

解析：欧拉序长度为 $2n - 1$ ，数组至少需 $2N$ 空间。

32. ② 处应填

答案：A `euler[timer] = u;` (3 分)

解析：`++timer` 已执行，当前位置直接写入节点 u 。

33. ③ 处应填

答案：D `euler[++timer] = u;` (3 分)

解析：离开节点 u 返回父节点前，需再次记录 u ；`timer` 需先自增。

34. ④ 处应填

答案：D `sort(to[u].begin(), to[u].end());` (3 分)

解析：对 `vector` 排序需使用迭代器 `begin()` / `end()`。

35. ⑤ 处应填

答案：C $i < 2 * n - 1$ (3 分)

解析：循环输出前 $2n - 2$ 个元素带空格，最后一个单独输出避免行尾空格。

程序二（第 36~40 题）：中国邮路问题

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
typedef long long ll;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> > dst(n + 1, vector<int>(n + 1, INT_MAX));
    vector<int> deg(n + 1, 0);
    ll total = 0;
    for (int i = 1; i <= n; ++i) dst[i][i] = 0;

    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        if (①) {
            dst[u][v] = w;
            dst[v][u] = w;
        }
        deg[u]++;
        deg[v]++;
        total += w;
    }

    // Floyd-Warshall
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i) {
            if (dst[i][k] == INT_MAX) continue;
            for (int j = 1; j <= n; ++j) {
                if (dst[k][j] == INT_MAX) continue;
                if (②)
                    dst[i][j] = dst[i][k] + dst[k][j];
            }
        }

    vector<int> odd;
    for (int i = 1; i <= n; ++i)
        if (deg[i] % 2 == 1) odd.push_back(i);
    if (③) {
        cout << total << endl;
        return 0;
    }
```



```

int k = odd.size();
vector<long long> dp(1 << k, LLONG_MAX);
dp[0] = 0;

for (int mask = 0; mask < (1 << k); ++mask) {
    if (dp[mask] == LLONG_MAX) continue;
    int i = 0;
    while (④) i++;
    if (i >= k) continue;
    for (int j = i + 1; j < k; ++j) {
        if (mask & (1 << j)) continue;
        if (dst[odd[i]][odd[j]] == INT_MAX) continue;
        int new_mask = ⑤;
        ll new_val = dp[mask] + dst[odd[i]][odd[j]];
        if (new_val < dp[new_mask]) dp[new_mask] = new_val;
    }
}

ll ans = total + dp[(1 << k) - 1];
cout << ans << endl;
return 0;
}

```

36. ① 处应填

答案: **A** `w < dst[u][v]` (3 分)

解析: 若有多重边, 保留权值最小的边。

37. ② 处应填

答案: **C** `dst[i][j] > dst[i][k] + dst[k][j]` (3 分)

解析: Floyd 松弛条件: 若经 k 中转更短则更新。

38. ③ 处应填

答案: **A** `odd.empty()` (3 分)

解析: 无奇度点则图是欧拉图, 直接输出所有边权和 `total`。

39. ④ 处应填

答案: **C** `i < k && (mask & (1 << i))` (3 分)

解析: 寻找第一个未匹配的顶点; 位为 1 表示已匹配, 继续向后找。

40. ⑤ 处应填

答案: **D** `mask | (1 << i) | (1 << j)` (3 分)

解析: 将顶点 i 和 j 都标记为已匹配, 得到新状态掩码。

答案速查表

题号	答案	分值	题号	答案	分值	题号	答案	分值	题号	答案	分值
1	B	2	11	A	2	21	A	2	31	D	3
2	A	2	12	B	2	22	B	2	32	A	3

题号	答案	分值	题号	答案	分值	题号	答案	分值	题号	答案	分值
3	B	2	13	B	2	23	B	2	33	D	3
4	C	2	14	B	2	24	D	4	34	D	3
5	D	2	15	B	2	25	A	4	35	C	3
6	C	2	16	A	2	26	A	2	36	A	3
7	C	2	17	B	2	27	A	2	37	C	3
8	D	2	18	A	2	28	A	2	38	A	3
9	B	2	19	D	3	29	D	4	39	C	3
10	A	2	20	B	3	30	C	4	40	D	3

CSP/NOI 模拟试卷解析

一、单项选择题（每题 2 分）

第 1 题

题目：在 NOI Linux 系统中，使用 g++ 编译 C++ 程序时，如果要生成可调试的可执行文件，应该使用选项

选项：`-O2` / `-g` / `-static` / `-Wall`

答案：B `-g`

解析：`-g` 选项生成调试信息，供 GDB 等调试器使用。

第 2 题

题目：执行 `int x = 0xAB; cout << (x >> 4 | (x & 0x0F) << 4);`，输出为

选项：186 / 171 / 180 / 176

答案：A 186

解析： $0xAB = 10101011_2$ ， $x \gg 4 = 0x0A$ ， $(x \& 0x0F) \ll 4 = 0xB0$ ，按位或得 $0xBA = 186$ 。

第 3 题

题目：连接 n 个字符串，每个字符串的长度不大于 m ，最坏情况下时间复杂度是

选项： $O(nm)$ / $O(n^2m)$ / $O(nm^2)$ / $O(\log n)$

答案：B $O(n^2m)$

解析：朴素拼接每次需复制已连接部分，总复制量约为 $m(1 + 2 + \cdots + n) = O(n^2m)$ 。

第 4 题

题目：已知某完全二叉树的层次遍历序列为 `ACBGFED`，则其中序遍历序列是

选项：`DBEAFCG` / `ABDCEFG` / `GCFAEBD` / `GCFAEDB`

答案：C GCFAEBD

解析：按层建树后中序遍历（左-根-右）得到 G C F A E B D。

第 5 题

题目：用两个栈模拟队列，如果 push 操作总是入栈 S_1 ，那么 pop 操作是

选项：

- A. 直接弹出 S_1 栈顶
- B. 直接弹出 S_2 栈顶
- C. 将 S_2 全部弹出并压入 S_1 ，然后弹出 S_1 栈顶
- D. 如果 S_2 为空，将 S_1 全部弹出并压入 S_2 ，然后弹出 S_2 栈顶

答案：D

解析：经典双栈模拟队列： S_1 负责入队， S_2 负责出队；仅当 S_2 为空时才将 S_1 倒入 S_2 。

第 6 题

题目：关于树的重心，下列说法中不正确的是

选项：

- A. 树至少有一个重心
- B. 树最多有两个重心，且这两个重心一定相邻
- C. 树最多有两个重心，且这两个重心可以不相邻
- D. 重心到其他点的距离之和最小

答案：C

解析：树的重心有 1 个或 2 个；若有两个，它们**必定相邻**。因此“可以不相邻”是错误的。

第 7 题

题目：给定数字 0、1、5、6、7、9，每个数字最多用一次，可能组成多少个正偶数

选项：216 / 480 / 586 / 589

答案：C 586

解析：按位数分类（末位为 0 或 6）：

1 位：1 个；2 位：9 个；3 位：36 个；4 位：108 个；5 位：216 个；6 位：216 个。

总计 $1 + 9 + 36 + 108 + 216 + 216 = 586$ 。

第 8 题

题目：最长公共子序列（LCS）的应用不包括

选项：文件差异比较 / DNA 序列比对 / 拼写检查 / 数据加密

答案：D 数据加密

解析：LCS 常用于 diff、生物序列比对、拼写检查等，与数据加密无关。

第 9 题

题目：给一个无向图着色，相邻节点不能同色，如果至少需要 3 种颜色，则该图

选项：一定是二分图 / 一定包含奇环 / 一定是完全图 / 一定是连通图

答案：B 一定包含奇环

解析：二分图等价于 2-可着色，也等价于不含奇环。需要 3 种颜色说明不是二分图，必含奇环。

第 10 题

题目：同余方程组 $x \equiv 1 \pmod{3}$, $x \equiv 2 \pmod{5}$ 的解为

选项： $x \equiv 7 \pmod{15}$ / $x \equiv 8 \pmod{15}$ / $x \equiv 11 \pmod{15}$ / $x \equiv 13 \pmod{15}$

答案：A $x \equiv 7 \pmod{15}$

解析：设 $x = 3k + 1$ ，代入得 $3k \equiv 1 \pmod{5}$ ，解得 $k \equiv 2 \pmod{5}$ ，故 $x = 15m + 7$ 。

第 11 题

题目：下列几种排序算法中，在平均情况下，哪一种的时间复杂度与其他三种不同

选项：选择排序 / 堆排序 / 归并排序 / 快速排序

答案：A 选择排序

解析：选择排序平均 $O(n^2)$ ；其余三种平均均为 $O(n \log n)$ 。

第 12 题

题目：5 名同学错位排列（每人不坐自己位置），共有多少种坐法

选项：36 / 44 / 48 / 120

答案：B 44

解析：错位排列 $!5 = 44$ 。

第 13 题

题目：哈希表长度 13, $H(key) = key \bmod 13$ ，线性探查。已插入 {26, 39, 52, 65, 78, 91, 104, 117}，查找 91 的探查下标依次是

选项：0, 1, 2, 3, 4 / 0, 1, 2, 3, 4, 5 / 0, 1, 2, 3, 4, 5, 6 / 0, 1, 2, 3, 4, 5, 6, 7

答案：B 0, 1, 2, 3, 4, 5

解析：插入后位置：0→26, 1→39, 2→52, 3→65, 4→78, 5→91。从 0 开始线性探查，到 5 找到。

第 14 题

题目：以下代码执行后输出

```
int a = 0, b = 1;
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 1) a = a + b;
    else b = a + b;
}
cout << a << " " << b << endl;
```

选项：5 8 / 8 13 / 13 21 / 21 34

答案：B 8 13

解析：模拟得： $i=1(1,1), 2(1,2), 3(3,2), 4(3,5), 5(8,5), 6(8,13)$ 。

第 15 题

题目：抛掷一枚均匀骰子，直到出现 2 次 6 为止，期望的抛掷次数是

选项：6 / 12 / 24 / 48

答案：B 12

解析：负二项分布，成功概率 $p = 1/6$ ，需 $r = 2$ 次成功，期望 $r/p = 12$ 。

二、阅读程序

程序一（第 16~20 题）：Legendre 公式求阶乘素因子幂次

```
#include <iostream>
using namespace std;
int main() {
    int n, p;
    cin >> n >> p;
    int ans = 0;
    long long cur = p;
    while (cur <= n) {
        ans += n / cur;
        cur *= p;
    }
    cout << ans << endl;
    return 0;
}
```

16. 输入 10 2，输出为 8。

答案：A 正确（2 分）

解析： $\lfloor 10/2 \rfloor + \lfloor 10/4 \rfloor + \lfloor 10/8 \rfloor = 5 + 2 + 1 = 8$ 。

17. 输出一定是正整数。

答案：B 错误（2 分）

解析：若 $n < p$ （如 $n=1, p=2$ ），循环不执行，输出 0。

18. 程序的时间复杂度为 $O(\log_p n)$ 。

答案：A 正确（2 分）

解析： cur 每次乘 p ，循环次数为 $\log_p n$ 。

19. 若希望从低位到高位依次输出 n 在 p 进制下的各位，应改为

答案：D `cout << n * p / cur % p << endl;`（3 分）

解析： cur 依次取 $p, p^2, \dots$ 。 $n * p / cur$ 等价于 $n / (cur/p)$ ，当 $cur=p$ 时得到 $n \bmod p$ （最低位），随后依次得到更高位。

20. 输入 $n=100$, $p=5$, 输出为

答案: B 24 (3 分)

解析: $\lfloor 100/5 \rfloor + \lfloor 100/25 \rfloor = 20 + 4 = 24$ 。

程序二 (第 21~25 题): 线性筛预处理约数和

```
#include <iostream>
using namespace std;

const int N = 100009;
int prime[N], tot, sd[N], sp[N];
bool vis[N];

void init() {
    sd[1] = 1;
    for (int i = 2; i < N; i++) {
        if (!vis[i]) {
            prime[++tot] = i;
            sd[i] = i + 1;
            sp[i] = i + 1;
        }
        for (int j = 1; j <= tot && i * prime[j] < N; j++) {
            vis[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                sp[i * prime[j]] = sp[i] * prime[j] + 1;
                sd[i * prime[j]] = sd[i] / sp[i] * sp[i * prime[j]];
                break;
            } else {
                sp[i * prime[j]] = prime[j] + 1;
                sd[i * prime[j]] = sd[i] * sd[prime[j]];
            }
        }
    }
}

int main() {
    init();
    int n;
    cin >> n;
    cout << sd[n] << endl;
    return 0;
}
```

21. 输入 7, 输出是 8。

答案: A 正确 (2 分)

解析: 7 是素数, 约数和 $1 + 7 = 8$ 。

22. $vis[i]$ 为 1 时, 表示 i 是素数。

答案: B 错误 (2 分)

解析: `vis[i]` 为 true 表示 i 是合数 (已被标记)。

23. `sd` 记录的是所有素因数之和。

答案: B 错误 (2 分)

解析: `sd` 记录的是所有约数之和 $\sigma(n)$, 不是素因数之和。

24. 输入 2026 时, 输出为

答案: D 3042 (4 分)

解析: $2026 = 2 \times 1013$ (1013 为素数), $\sigma(2026) = (1 + 2)(1 + 1013) = 3 \times 1014 = 3042$ 。

25. `init()` 的时间复杂度是

答案: A $O(n)$ (4 分)

解析: 线性筛每个合数仅被其最小质因子筛去一次, 复杂度 $O(n)$ 。

程序三 (第 26~30 题): 费马小定理求逆元

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
ll T, a, m, ans;

ll quickpow(ll a, ll b) {
    if (b < 0) return 0;
    ll ret = 1;
    a %= m;
    while (b) {
        if (b & 1) ret = (ret * a) % m;
        b >>= 1;
        a = (a * a) % m;
    }
    return ret;
}

ll inverse(ll a, ll m) {
    return quickpow(a, m - 2);
}

int main() {
    cin >> a >> m;
    ans = inverse(a, m);
    cout << ans << " ";
    return 0;
}
```

26. 输入 `a=3, m=7`, 输出是 5。

答案: A 正确 (2 分)

解析: $3^5 \bmod 7 = 243 \bmod 7 = 5$ 。

27. 输入 $a=4$, $m=6$, 输出是 4。

答案: A 正确 (2 分)

解析: 程序计算 $4^{6-2} = 4^4 \pmod{6}$ 。快速幂过程中 $4^2 \equiv 4 \pmod{6}$, 故 $4^4 \equiv 4 \pmod{6}$, 程序确实输出 4。(注: 虽然 $\gcd(4, 6) \neq 1$ 导致逆元不存在, 但题目仅判断输出值。)

28. 输入 $m=2$, 无论 a 为何值, 输出都是 1。

答案: A 正确 (2 分)

解析: $m = 2$ 时指数 $m - 2 = 0$, 快速幂循环不执行, 直接返回 `ret = 1`。

29. 程序的时间复杂度是

答案: D $O(\log m)$ (4 分)

解析: 快速幂复杂度取决于指数 $m - 2$ 的二进制位数。

30. 关于 `inverse()` 函数, 说法正确的是

答案: C 它使用费马小定理, 要求 m 必须是素数 (4 分)

解析: $a^{m-2} \equiv a^{-1} \pmod{m}$ 是费马小定理的直接推论, 要求 m 为素数且 $a \not\equiv 0 \pmod{m}$ 。

三、完善程序

程序一 (第 31~35 题): 树的欧拉序 (Euler Tour)

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1009;
vector<int> to[N];
int n, timer, euler[0];

void dfs(int u, int fa) {
    ++timer;
    ②
    for (int i = 0; i < to[u].size(); ++i)
        if (to[u][i] != fa) dfs(to[u][i], u);
    ③
}

int main() {
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int u, v;
        cin >> u >> v;
        to[u].push_back(v);
        to[v].push_back(u);
    }
    for (int u = 1; u <= n; ++u)
        ④
    dfs(1, 0);
    for (int i = 1; ⑤; ++i) cout << euler[i] << " ";
    cout << euler[2 * n - 1] << endl;
}
```



```
    return 0;
}
```

31. ① 处应填

答案：D $N + N$ (3 分)

解析：欧拉序长度为 $2n - 1$ ，数组至少需 $2N$ 空间。

32. ② 处应填

答案：A `euler[timer] = u;` (3 分)

解析：`++timer` 已执行，当前位置直接写入节点 u 。

33. ③ 处应填

答案：D `euler[++timer] = u;` (3 分)

解析：离开节点 u 返回父节点前，需再次记录 u ；`timer` 需先自增。

34. ④ 处应填

答案：D `sort(to[u].begin(), to[u].end());` (3 分)

解析：对 `vector` 排序需使用迭代器 `begin()` / `end()`。

35. ⑤ 处应填

答案：C `i < 2 * n - 1` (3 分)

解析：循环输出前 $2n - 2$ 个元素带空格，最后一个单独输出避免行尾空格。

程序二（第 36~40 题）：中国邮路问题

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
typedef long long ll;

int main() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> > dst(n + 1, vector<int>(n + 1, INT_MAX));
    vector<int> deg(n + 1, 0);
    ll total = 0;
    for (int i = 1; i <= n; ++i) dst[i][i] = 0;

    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        if (①) {
            dst[u][v] = w;
            dst[v][u] = w;
        }
        deg[u]++;
        deg[v]++;
        total += w;
    }
}
```

```

    }

    // Floyd-Warshall
    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i) {
            if (dst[i][k] == INT_MAX) continue;
            for (int j = 1; j <= n; ++j) {
                if (dst[k][j] == INT_MAX) continue;
                if (②)
                    dst[i][j] = dst[i][k] + dst[k][j];
            }
        }

    vector<int> odd;
    for (int i = 1; i <= n; ++i)
        if (deg[i] % 2 == 1) odd.push_back(i);
    if (③) {
        cout << total << endl;
        return 0;
    }

    int k = odd.size();
    vector<long long> dp(1 << k, LLONG_MAX);
    dp[0] = 0;

    for (int mask = 0; mask < (1 << k); ++mask) {
        if (dp[mask] == LLONG_MAX) continue;
        int i = 0;
        while (④) i++;
        if (i >= k) continue;
        for (int j = i + 1; j < k; ++j) {
            if (mask & (1 << j)) continue;
            if (dst[odd[i]][odd[j]] == INT_MAX) continue;
            int new_mask = ⑤;
            ll new_val = dp[mask] + dst[odd[i]][odd[j]];
            if (new_val < dp[new_mask]) dp[new_mask] = new_val;
        }
    }

    ll ans = total + dp[(1 << k) - 1];
    cout << ans << endl;
    return 0;
}

```

36. ① 处应填

答案: $A_w < \text{dst}[u][v]$ (3 分)

解析: 若有多重边, 保留权值最小的边。

37. ② 处应填

答案: $C \text{dst}[i][j] > \text{dst}[i][k] + \text{dst}[k][j]$ (3 分)

解析: Floyd 松弛条件: 若经 k 中转更短则更新。

38. ③ 处应填

答案：A `odd.empty()` (3 分)

解析：无奇度点则图是欧拉图，直接输出所有边权和 `total`。

39. ④ 处应填

答案：C `i < k && (mask & (1 << i))` (3 分)

解析：寻找第一个未匹配的顶点；位为 1 表示已匹配，继续向后找。

40. ⑤ 处应填

答案：D `mask | (1 << i) | (1 << j)` (3 分)

解析：将顶点 i 和 j 都标记为已匹配，得到新状态掩码。

答案速查表

题号	答案	分值	题号	答案	分值	题号	答案	分值	题号	答案	分值
1	B	2	11	A	2	21	A	2	31	D	3
2	A	2	12	B	2	22	B	2	32	A	3
3	B	2	13	B	2	23	B	2	33	D	3
4	C	2	14	B	2	24	D	4	34	D	3
5	D	2	15	B	2	25	A	4	35	C	3
6	C	2	16	A	2	26	A	2	36	A	3
7	C	2	17	B	2	27	A	2	37	C	3
8	D	2	18	A	2	28	A	2	38	A	3
9	B	2	19	D	3	29	D	4	39	C	3
10	A	2	20	B	3	30	C	4	40	D	3